

## OPERATING SYSTEMS (UE25CS242A)

### Unit 1 – CPU Scheduling

#### CPU Scheduling – Basic Concepts

In a computer system with a **single CPU core**, only **one process can execute at a time**. All other processes must wait until the CPU becomes free and they are scheduled again.

The main objective of **multiprogramming** is to:

- Keep the CPU busy at all times
- Maximize CPU utilization

To achieve this:

- Several processes are kept in main memory
- When one process has to wait (usually for I/O), the OS:
  - Takes the CPU away from that process
  - Assigns the CPU to another ready process

This cycle continues, ensuring that the CPU rarely remains idle.

#### CPU Scheduling in Multicore Systems

On a **multicore system**, the same concept is extended:

- Each processing core should be kept busy
- Scheduling decisions are made independently for each core
- Improves overall system throughput and performance

#### CPU Scheduling – Need for Multiprogramming

In a simple system without multiprogramming:

- A process executes until it must wait (typically for I/O)
- During this waiting period, the CPU remains idle
- CPU time is wasted and no useful work is done

Multiprogramming allows the OS to:

- Switch the CPU to another ready process
- Use waiting time productively

This makes **CPU scheduling a fundamental OS function**.

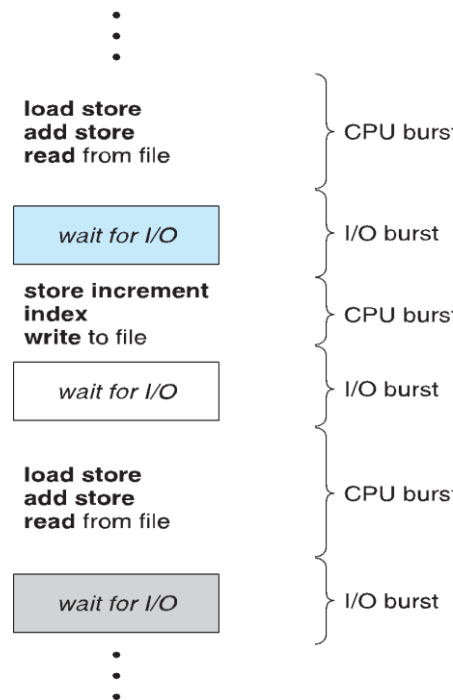
#### Alternating Sequence of CPU and I/O Bursts

Process execution consists of an **alternating sequence of CPU bursts and I/O bursts**.

- **CPU burst** → process is executing instructions
- **I/O burst** → process is waiting for I/O completion

Maximum CPU utilization is achieved when:

- CPU bursts of one process overlap with I/O bursts of another



### CPU-I/O Burst Cycle

- Execution pattern:
  - CPU burst → I/O burst → CPU burst → I/O burst → ...
- CPU scheduling algorithms focus mainly on:
  - The **distribution of CPU burst times**

Understanding this cycle is essential for designing efficient schedulers.

### Histogram of CPU-Burst Times

The duration of CPU bursts:

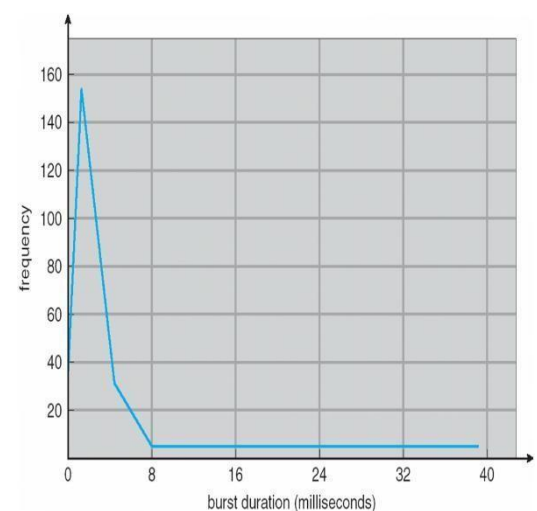
- Varies widely among processes
- Follows a characteristic frequency distribution

### Observations

- **I/O-bound processes**
  - Many short CPU bursts
- **CPU-bound processes**
  - Few long CPU bursts

This distribution is important when:

- Designing
- Evaluating
- Comparing CPU scheduling algorithms



## CPU Scheduler

The **CPU scheduler** (short-term scheduler):

- Selects a process from the **ready queue**
- Allocates the CPU to that process

## Ready Queue Organization

The ready queue may be organized in various ways:

- FIFO queue
- Priority queue
- Tree
- Unordered linked list

Each entry in the queue is a **Process Control Block (PCB)**.

## Preemptive Scheduling

CPU scheduling decisions can occur when a process:

1. Switches from **running** → **waiting**
2. Switches from **running** → **ready**
3. Switches from **waiting** → **ready**
4. **Terminates**

## Scheduling Types

- Cases **1 and 4** → **Non-preemptive**
- Cases **2 and 3** → **Preemptive**

## Issues in Preemptive Scheduling

When preemption occurs:

- Shared data access must be handled carefully
- Preemption while executing in kernel mode must be considered
- Interrupts may occur during critical OS operations

## Examples

- Windows 3.x → Non-preemptive
- Windows 95 onwards → Preemptive
- Macintosh OS → Uses preemptive scheduling

## Preemptive vs Non-Preemptive Scheduling

## Problems with Preemptive Scheduling

- Can cause **race conditions**

- Occurs when:
  - One process is preempted while updating shared data
  - Another process accesses inconsistent data

### Solution

- Preemptive kernels use:
  - **Mutex locks**
  - Other synchronization mechanisms

Most modern operating systems are:

- **Fully preemptive**, even in kernel mode

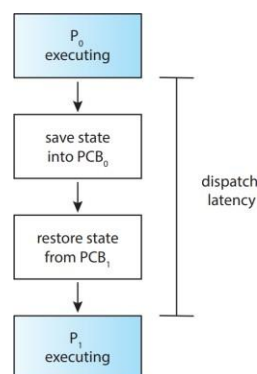
### Dispatcher

The **dispatcher** is a module that:

- Gives control of the CPU to the process selected by the short-term scheduler

### Dispatcher Functions

- Performs **context switch**
- Switches CPU to **user mode**
- Jumps to the correct location in the user program



**Fig: The role of the dispatcher.**

### Dispatch Latency

- Time taken to:
  - Stop one process
  - Start another process
- Important performance metric

### Scheduling Criteria

Scheduling algorithms are evaluated based on the following criteria:

#### 1. CPU Utilization

- Percentage of time CPU is busy

## 2. **Throughput**

- Number of processes completed per unit time

## 3. **Turnaround Time**

- Time from submission to completion of a process

## 4. **Waiting Time**

- Total time a process spends waiting in the ready queue

## 5. **Response Time**

- Time from request submission to first response
- Important in time-sharing systems

### **Scheduling Algorithm Optimization Criteria**

An ideal scheduling algorithm should aim to:

- **Maximize**
  - CPU utilization
  - Throughput
- **Minimize**
  - Turnaround time
  - Waiting time
  - Response time

Different scheduling algorithms optimize different criteria based on system goals.

## Unit 1 – CPU Scheduling Algorithms – FCFS and SJF

### Scheduling Algorithms

Scheduling algorithms decide which process gets the CPU and for how long. In this lecture, we study:

- **First-Come, First-Served (FCFS)**
- **Shortest-Job-First (SJF)** and its preemptive form

### First-Come, First-Served (FCFS) Scheduling FCFS

is the simplest CPU scheduling algorithm.

### Characteristics

- Processes are executed in the order they arrive
- Uses a FIFO queue
- Non-preemptive algorithm
- Once CPU is allocated, the process runs until completion or I/O wait

### FCFS Example – Case 1

Suppose the processes arrive in the order: **P<sub>1</sub>, P<sub>2</sub>, P<sub>3</sub>**

| Process        | Burst Time |
|----------------|------------|
| P <sub>1</sub> | 24         |
| P <sub>2</sub> | 3          |
| P <sub>3</sub> | 3          |



### Waiting Time

- Waiting time of P<sub>1</sub> = 0
- Waiting time of P<sub>2</sub> = 24
- Waiting time of P<sub>3</sub> = 27

### Average Waiting Time

$$(0 + 24 + 27)/3 = 17$$

## FCFS Example – Case 2

Suppose the processes arrive in the order:  $P_2, P_3, P_1$



### Waiting Time

- Waiting time of  $P_2 = 0$
- Waiting time of  $P_3 = 3$
- Waiting time of  $P_1 = 6$

### Average Waiting Time

$$(6 + 0 + 3)/3 = 3$$

### Observation

- FCFS **does not guarantee minimum average waiting time**
- Performance varies greatly when CPU burst times differ significantly

### Drawbacks of FCFS Scheduling Convoy

#### Effect

- Short processes get stuck **behind a long process**
- Example:
  - One CPU-bound process
  - Many I/O-bound processes

### Additional Issues

- FCFS is **non-preemptive**
- A process can occupy the CPU for a long time
- Poor choice for **time-sharing systems**
- Not fair when responsiveness is important

### Shortest-Job-First (SJF) Scheduling

SJF associates each process with the **length of its next CPU burst**. Scheduling

#### Rule

- Process with the **shortest CPU burst** is scheduled next

### Key Properties

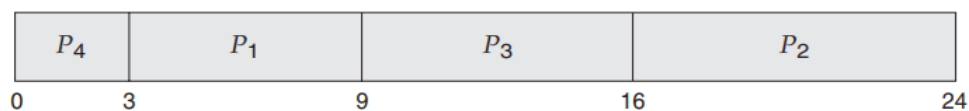
- Optimal scheduling algorithm
- Produces minimum average waiting time
- Can be:
  - Non-preemptive (SJF)
  - Preemptive (Shortest-Remaining-Time-First)

### Main Challenge

- Exact length of next CPU burst is **not known in advance**
- Must be **predicted**

Example of SJF Scheduling

| Process | Burst Time |
|---------|------------|
| P1      | 6          |
| P2      | 8          |
| P3      | 7          |
| P4      | 3          |



### Average Waiting Time (SJF)

$$(3 + 16 + 9 + 0)/4 = 7$$

### Comparison with FCFS

If FCFS is used:

$$(0 + 6 + 14 + 21)/4 = 10.25$$

### Observation

- SJF gives better (lower) average waiting time than FCFS

### Determining Length of Next CPU Burst

Since actual burst time is unknown, OS predicts it using **exponential averaging**.



## Formula

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

Where:

- $\tau_{n+1} \rightarrow$  predicted next CPU burst
- $t_n \rightarrow$  actual last CPU burst
- $\tau_n \rightarrow$  previous prediction
- $\alpha \rightarrow$  weighting factor ( $0 \leq \alpha \leq 1$ )

## Notes

- $\alpha$  controls importance of recent history
- Commonly  $\alpha = 0.5$
- SJF preemptive version is called **Shortest-Remaining-Time-First (SRTF)**

## Numerical Example – Exponential Averaging

Given:

- $T_1 = 10$
- $\alpha = 0.5$
- Previous CPU bursts: **8, 7, 4, 16**

Since SJF is non-preemptive, order becomes: **4, 7, 8, 16 Calculations**

- $T_2 = 0.5 \times 4 + 0.5 \times 10 = 7$
- $T_3 = 0.5 \times 7 + 0.5 \times 7 = 7$
- $T_4 = 0.5 \times 8 + 0.5 \times 7 = 7.5$
- $T_5 = 0.5 \times 16 + 0.5 \times 7.5 = 11.8$

## Result

- **Predicted 5<sup>th</sup> CPU burst = 11.8**

## Prediction Behavior with Different $\alpha$ Values

### Special Cases

- $\alpha = 0$ 
  - $\tau_{n+1} = \tau_n$
  - Recent burst ignored
- $\alpha = 1$ 
  - $\tau_{n+1} = t_n$
  - Only last burst considered

## Expanded Formula

$$\tau_{n+1} = \alpha \tau_n + (1 - \alpha) \alpha \tau_{n-1} + \dots + (1 - \alpha)^{n+1} \tau_0$$

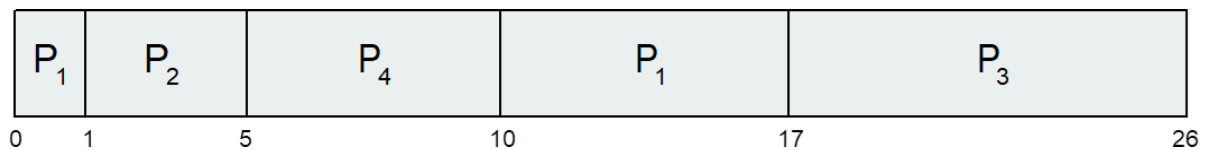
Each older burst has **less weight**, ensuring stability.

## Shortest-Remaining-Time-First (SRTF)

- **Preemptive version of SJF**
- If a new process arrives with **shorter remaining burst time**, it preempts the running process

SRTF Example

| <u>Process</u> | <u>Arrival Time</u> | <u>Burst Time</u> |
|----------------|---------------------|-------------------|
| $P_1$          | 0                   | 8                 |
| $P_2$          | 1                   | 4                 |
| $P_3$          | 2                   | 9                 |
| $P_4$          | 3                   | 5                 |



## Average Waiting Time

$$[(10 - 1) + (1 - 1) + (17 - 2) + (5 - 3)]/4 = 6.5 \text{ ms}$$